



Requirement & Design Specification
Crowndine - Restaurant Management System

Subject: SWD392

Version: 1.0

Record of Changes

Version	Date	In Charge	Change Description
1.0	July 2026	Project Team	Initial version

* A - Added M - Modified D - Deleted

Table of Contents

Record of Changes	i
I Requirement Specification	1
I.1 Problem Description	1
I.2 Major Features	1
I.2.1 Features for Customers	1
I.2.2 Features for Staff	3
I.2.3 Features for Admin	3
I.3 System Context	4
I.4 Non-Functional Requirements	5
I.5 Functional Requirements	6
I.5.1 Use Case Diagrams	6
I.5.2 Use Case Descriptions	6
I.5.3 Activity Diagrams	6
I.6 Data Requirements	6
II Analysis Models	6
II.1 Interaction Diagrams	6
II.2 State Diagram	7
III Design Specification	7
III.1 Integrated Communication Diagrams	7
III.2 System High-Level Design	7
III.3 Component & Package Diagram	7
III.4 Detailed Design	7
III.5 Database Design	7
IV Implementation	7
IV.1 Project Architecture	7
IV.2 Mapping Class Diagrams & Interaction Diagrams to Code	8
IV.3 Applying Alternative Architecture Patterns	8
V Service-Oriented Architecture (SOA)	8
V.1 Service Discovery Pattern	8

I Requirement Specification

I.1 Problem Description

In today's Food and Beverage (F&B) industry, restaurants continue to face challenges in optimizing operational processes, including table reservations, order management, kitchen coordination, and payment handling, due to the lack of an integrated and efficient management system.

The main problems are described as follows:

- **Inefficient reservation and ordering experience:** Customers often have limited flexibility when selecting their preferred tables and placing orders in advance. As a result, restaurants face difficulties in preparing ingredients, allocating seating arrangements, and anticipating customer demand effectively.
- **Operational communication challenges:** Information exchange between service staff, kitchen staff, and cashiers can be slow or prone to errors. Miscommunication may lead to incorrect orders, delays in food preparation, and reduced service quality, particularly during peak business hours.
- **Limited decision-support capabilities:** Restaurant managers often struggle to consolidate and analyze business data from multiple sources. This makes it difficult to identify customer trends, optimize menu offerings, improve inventory management, and make data-driven business decisions.
- **Inefficient resource and inventory management:** Without a centralized system, monitoring ingredient consumption, tracking stock levels, and managing restaurant resources can be time-consuming and inaccurate, potentially leading to waste and increased operational costs.

CrownDine is designed to address these challenges through a centralized restaurant management platform that streamlines reservation, ordering, kitchen coordination, payment processing, and business management activities. Additionally, the system integrates Artificial Intelligence (AI) to provide valuable insights and support smarter business decisions, helping restaurants improve operational efficiency and enhance customer satisfaction.

I.2 Major Features

The CrownDine system provides separate feature groups for customers, staff, and administrators. These features are adapted from the implemented business logic in both the Backend and Frontend, including reservation workflows, online payment, restaurant operations, management dashboards, and AI chatbot integrations.

I.2.1 Features for Customers

F01: Viewing Menu & Product Details

- Browse food items and drinks grouped by categories.

- Browse food combo packages.
- View detailed information of items and combos, including descriptions, prices, images, and customer feedback.
- Search items and combos by name and filter them by categories.
- Add items to the favorites list, view favorite items, and manage the favorites list.

F02: Smart Table Reservation

- Use a step-by-step reservation wizard.
- Select booking date, dining time slot, and guest count.
- View an interactive table map showing table locations, floor layouts, table status, shape, and seating capacity.
- Select preferred tables for the reservation.

F03: Pre-order Cart Management

- Add food items and combos to the reservation order during booking.
- View the food reservation cart.
- Edit cart items by changing quantities or removing items.

F04: Booking Placement & Payment

- Review reservation details and total price before checkout.
- Apply discount vouchers.
- Make online payments through PayOS integration using QR Code or bank transfer.
- Receive real-time payment success or failure redirection and booking confirmation.
- Cancel pending or upcoming bookings.

F05: Customer Authentication

- Register a new account with email verification via OTP.
- Log in using username and password or Google OAuth2.
- Securely log out, refresh login tokens, and update passwords.

F06: Profile & Loyalty Program

- View and update customer personal profile details.
- Request email update with OTP verification for enhanced security.
- Upload profile avatar images processed and hosted through Cloudinary.
- View reservation history, order details, and statuses.
- Track accumulated loyalty membership points.
- Exchange loyalty points for discount vouchers.

F07: Customer AI Booking Assistant

- Chat in real time with AI chatbot for booking assistance.
- Ask questions about the menu, dishes, prices, and opening hours.
- Automatically parse relative time expressions into actual calendar dates.
- View available table recommendations matching guest counts.
- Confirm table selection and receive direct routing links to step 3 for pre-order menu or step 4 for payment and checkout.

I.2.2 Features for Staff

F08: Table Reservation Processing

- View the full list of customer reservations and filter by date ranges and reservation statuses, including Pending, Confirmed, Checked-in, Completed, and Cancelled.
- Look up booking details and search reservations.
- Perform customer check-in when guests arrive at the restaurant.
- Reassign or update tables for an active reservation.
- Cancel reservation bookings for customers with required cancellation reasons.

F09: Cashier & Walk-in Dine-in Orders

- Create walk-in dine-in orders directly for a table.
- Retrieve customer profiles by phone number to link them to orders for point accumulation.
- Append items, modify quantities, or remove items in active dine-in orders.
- Apply vouchers to active orders.
- Merge tables or split bills.
- Generate PayOS checkout QR codes for bills, process payments, and track transactions.
- Print invoice receipts and close completed tables by setting table status to Free.

F10: Kitchen Display System & Order Processing

- Use a real-time Kitchen Display System (KDS) for kitchen staff to view pending and preparation order queues.
- Group identical ordered dishes into batch preparations.
- Update item status in real time from Pending to Preparing to Ready using WebSockets.
- Automatically notify floor staff through toast notifications when dishes are marked as Ready for serving.

F11: Shift Schedule & Attendance Tracking

- View assigned weekly work schedules.
- View personal attendance history.
- Perform check-in and check-out to log work hours.

I.2.3 Features for Admin

F12: Restaurant Layout & Table Design

- Add, edit, and delete floors and areas.
- Add, edit, and delete restaurant tables.
- Configure the visual layout by editing table names, coordinates, width, height, shape, and seat capacity to build an interactive floor plan.

F13: Menu & Category Management

- Add, edit, delete, and search food categories.
- Add, edit, delete, and search food items and combo packages.

- Manage item pricing, upload dish images through Cloudinary integration, write descriptions, and update combo item lists.

F14: Promotion & Voucher Management

- Create, edit, and delete discount vouchers.
- Configure voucher types, including percentage discounts and fixed amounts, minimum order values, expiry dates, usage limits, and descriptions.
- Manually assign vouchers to specific users.
- Manage point exchange rates and point costs for vouchers.

F15: Staff Account & Role Assignment

- Create new staff accounts.
- Search staff profiles by name and view detailed staff information.
- Update staff profiles, edit status as Active or Blocked, and assign system roles and authorities such as Admin, Staff, Kitchen, and Cashier.

F16: Employee Shift Scheduling

- Configure daily shifts, such as Morning, Afternoon, and Evening.
- Assign weekly work schedules to staff members.
- Update or reassign scheduled shifts, change work schedule status, and delete schedules.

F17: Attendance Auditing & Payroll Summary

- View weekly schedules and clock-in statuses for all staff.
- Track detailed clock-in logs.
- View monthly attendance summaries.

F18: Sales Analytics Dashboard & Excel Export

- View interactive dashboard sales graphs, including revenue by hour or day, customer counts, and growth rates.
- Export comprehensive sales reports to Excel spreadsheets.

I.3 System Context

The system context of the CrownDine Restaurant Management System illustrates the interactions between the system and its entities, including external users and external systems.

External Users:

- **Customer:** The system's end-user who can browse the digital menu, view food details, make table reservations, pre-order dishes, make payments, track booking status, manage personal profiles, and use the AI booking assistant.
- **Staff:** Responsible for handling restaurant operations, including managing reservations, checking in customers, creating walk-in orders, updating order information, processing bills, merging tables, splitting invoices, managing Kitchen Display System (KDS) orders, updating dish preparation status, and notifying when dishes are ready to serve.

- **Admin:** Manages the entire system, including restaurant layout, tables, menu items, categories, staff accounts, user roles, vouchers, staff schedules, attendance records, revenue reports, and AI-based business analytics.

External Systems:

- **PayOS Payment Gateway:** Handles online payment transactions through QR Code and bank transfer, and returns payment status to the CrownDine system.
- **Google Authentication Service:** Provides login services using users' Google accounts through Google OAuth2 authentication.
- **Google Gemini AI Service:** Provides AI chatbot and data analysis functions, including revenue analysis, popular dish analysis, customer trend analysis, and demand prediction support.
- **Cloudinary Image Storage Service:** Stores and delivers uploaded images, including food images, combo images, and user avatar images.
- **Mailing / Notification System:** Sends OTP verification, booking confirmations, payment notifications, promotional messages, and real-time operational alerts to users.

I.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes and operating constraints that the Crowndine Restaurant Management System must satisfy. These requirements ensure that the system is not only functionally correct but also practical, secure, reliable, and maintainable in a real restaurant environment.

- **Usability:** The system shall provide an intuitive, seamless experience for all roles. For customers, the 4-step booking journey, interactive Table Map, and vibrant digital menu must be highly accessible. For staff and kitchen personnel, the Calendar View and KDS interfaces must be visually clear to manage real-time operations without requiring extensive technical training.
- **Reliability:** CrownDine must guarantee absolute accuracy in preserving reservations, order batching sequences, and financial records processed via PayOS (QR Code and Bank Transfer). The system must consistently maintain state synchronization across all modules and reliably execute the automated cancellation and 80% refund policy without any data loss or manual intervention.
- **Performance:** The platform shall process core actions—such as interactive table selection, cart updates, and payment authorizations—with minimal latency. The Kitchen Display System (KDS) must efficiently utilize WebSockets to ensure instant, real-time status updates under heavy loads, while the integrated Gemini AI chatbot must deliver prompt analytical insights.
- **Security:** All user data and transactions shall be strictly protected. The system must implement robust authentication utilizing JWT and Google OAuth2. Furthermore, a granular role-based access control (RBAC) mechanism must precisely isolate permissions among Admin, Staff, Kitchen, and Customer roles to prevent unauthorized system modifications.

- **Maintainability:** The software architecture shall be highly modular, enabling developers to seamlessly implement updates to the WebSockets infrastructure, PayOS integrations, and AI features without system downtime. This ensures easy future expansion and allows administrators to flexibly configure business rules via the admin panel rather than deep code changes.

I.5 Functional Requirements

FR1. The system shall allow customers to view menu items and restaurant information.

FR2. The system shall allow customers to create and manage reservations.

FR3. The system shall allow customers to place and track orders.

FR4. The system shall allow staff to confirm, update, and complete orders.

FR5. The system shall allow administrators to manage users, menu items, and reports.

I.5.1 Use Case Diagrams

This section should be completed by the project team with project-specific details, diagrams, tables, and references for the Crowndine Restaurant Management System.

I.5.2 Use Case Descriptions

I.5.3 Activity Diagrams

This section should be completed by the project team with project-specific details, diagrams, tables, and references for the Crowndine Restaurant Management System.

I.6 Data Requirements

Core data entities include User, Customer, Staff, Menu Item, Category, Reservation, Table, Order, Order Detail, Payment, Feedback, and Notification.

II Analysis Models

II.1 Interaction Diagrams

This section should be completed by the project team with project-specific details, diagrams, tables, and references for the Crowndine Restaurant Management System.

II.2 State Diagram

The main order lifecycle may include Created, Confirmed, Preparing, Ready, Served, Completed, and Cancelled states. State transitions must be triggered only by authorized users or valid system events.

III Design Specification

III.1 Integrated Communication Diagrams

This section should be completed by the project team with project-specific details, diagrams, tables, and references for the Crowndine Restaurant Management System.

III.2 System High-Level Design

The system follows a layered architecture consisting of presentation, application service, domain logic, data access, and persistence layers. Each layer separates responsibilities to improve maintainability and testability.

III.3 Component & Package Diagram

This section should be completed by the project team with project-specific details, diagrams, tables, and references for the Crowndine Restaurant Management System.

III.4 Detailed Design

Detailed design shall define classes, interfaces, service contracts, validation rules, exception handling, and data transfer objects for each major module.

III.5 Database Design

The database design shall include an entity relationship diagram, table schemas, primary keys, foreign keys, constraints, indexes, and sample records where applicable.

IV Implementation

IV.1 Project Architecture

The implementation should organize modules by responsibility, such as user management, menu management, reservation management, order management, payment integration, notification, and reporting.

IV.2 Mapping Class Diagrams & Interaction Diagrams to Code

Class diagrams shall be mapped to source code classes, interfaces, models, repositories, controllers, and services. Interaction diagrams shall be mapped to method calls, API flows, and transaction boundaries.

IV.3 Applying Alternative Architecture Patterns

Alternative patterns such as Model–View–Controller, layered architecture, repository pattern, and dependency injection may be applied where appropriate to improve separation of concerns.

V Service-Oriented Architecture (SOA)

V.1 Service Discovery Pattern

The Service Discovery Pattern enables services to locate each other dynamically through a service registry. In the Crowndine system, this pattern may be applied if the project evolves into independent services such as order service, menu service, payment service, and notification service.